

Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 6ο : Συστήματα Μικροϋπολογιστών

Δημήτριος Θανόγιαννης – 03121902

2η Ομάδα Ασκήσεων

Ασκήσεις προσομοίωσης:

1η Άσκηση:

α) Κάνοντας χρήση εργαλείων online disassembler, ο κώδικας assembly ARM Thumb-2 που προκύπτει από το παραπάνω hex machine code είναι ο εξής:

```
MOVS R0, #10 // R0 = 10
MOVS R1, #3 // R1 = 3
LSLS R2, R0, #2 // R2 = 40
ADDS R2, R2, R1 // R2 = 43
MOVS R1, #5 // R1 = 5
SUBS R2, R2, R1 // R2 = 38
LDR R3, [PC, #4] // R3 => [PC + 4] (.word  0x20001000)
STR R2, [R3] // [R3] = 38
loop:
        B loop
.align 2 // align memory by word (4 Bytes)
.word 0x20001000 // 4 Bytes of data (1 word)
```

β) Το παραπάνω πρόγραμμα έχει την ακόλουθη λειτουργία. Αρχικοποιεί τον καταχωρητή R0 με την τιμή 10, τον R1 με την τιμή 3, και τον R2 με την τιμή R0 * 4 (R0 << 2 = R0 * 4). Στη συνέχεια προσθέτει στον καταχωρητή R2 την τιμή του R1, θέτει στον R1 την τιμή 5 και αφαιρεί από τον R2 τη νέα τιμή του R1. Φορτώνει στον R3 τη διεύθυνση 0x20001000 και αποθηκεύει στη διεύθυνση αυτή της μνήμης την τελική τιμή του R2. Ο κώδικας τελειώνει με ένα infinite loop χωρίς άλλη εκτέλεση εντολής.

Η εντολή LDR R3, [PC, #4] έχει μια ιδιαιτερότητα , καθώς χρησιμοποιεί σχετική ως προς τον Program Counter διευθυνσιοδότηση. Εφόσον έχουμε ARM Thumb-2 αρχιτεκτονική, οι εντολές στοιχίζονται στη μνήμη ανά 2 Byte. Εφόσον έχουμε ένα 3 stage pipeline (fetch, decode, execute), τη στιγμή που η εντολή αυτή γίνεται execute, ο PC θα δείχνει στην εντολή που γίνεται fetch, δηλαδή 2 εντολές παρακάτω (4 Bytes). Προσθέτοντας στον PC τον αριθμό 4, αυτός δείχνει σε 4 Bytes ακόμη παρακάτω, δηλαδή 8 Bytes από την εντολή LDR. Η οδηγία .align 2 είναι οδηγία ευθυγράμμισης της μνήμης σε words (4 Bytes). Αυτή χρειάζεται, διότι οι εντολές είναι 9 και το declaration .word δε θα είναι ευθυγραμμισμένο κατά 4 Bytes. Η αμέσως επόμενη θέση μνήμης είναι το δεδομένο (read only) .word 0x20001000, το οποίο θα φορτωθεί στον καταχωρητή R3.

γ) Το πρόγραμμα λειτουργεί σωστά, όπως φαίνεται από τα περιεχόμενα των καταχωρητών:

Name	Value
General Registers	
r0	10
r1	5
r2	38
r3	536875008

2η Άσκηση:

Ο κώδικας που υλοποιεί τη ζητούμενη λειτουργικότητα είναι ο ακόλουθος:

```
.syntax unified
.thumb
.text
.global asm_main
.type asm_main, %function
.align 2

.equ RCC_BASE,   0x40023800   // Reset and Clock Control
.equ GPIOA_BASE, 0x40020000   // GPIOA base (Output)
.equ GPIOD_BASE, 0x40020C00   // GPIOD base (Input)

.equ RCC_AHB1ENR, 0x30        // RCC AHB 1 enable
.equ MODER_OFFSET,  0x00      // Mode Register
.equ OTYPER_OFFSET, 0x04      // Output Type Register
.equ PUPDR_OFFSET,  0x0C      // Pull-Up / Pull-Down Register
.equ IDR_OFFSET,    0x10      // Input Data Register
.equ ODR_OFFSET,    0x14      // Output Data Register

asm_main:
    LDR R0, =RCC_BASE
    LDR R1, [R0, #RCC_AHB1ENR]
    ORR R1, R1, #0x09           // set bits 0 and 3 of RCC_AHB1ENR (A and D, respectively)
    STR R1, [R0, #RCC_AHB1ENR]  // we enable the clock for these ports

    LDR R0, =GPIOA_BASE

    LDR R1, =0x55555555 // set bits for all 16 pins to 01
    STR R1, [R0, #MODER_OFFSET] // so that they are used for the output

    MOV R1, #0x0000 // set bits for all 16 pins to 0
    STR R1, [R0, #OTYPER_OFFSET] // so that we have push-pull output
```

```
    LDR R0, =GPIOD_BASE

    LDR R1, [R0, #MODER_OFFSET]
    BIC R1, R1, #0xC0000000     // clear bits 31, 30 for MSB
    BIC R1, R1, #0x00000003     // and bits 1, 0 for LSB
    STR R1, [R0, #MODER_OFFSET] // leds in LSB and MSB are used for the input

    LDR R1, [R0, #PUPDR_OFFSET]
    BIC R1, R1, #0xC0000000     // clear bits 31, 30 for MSB
    BIC R1, R1, #0x00000003     // and bits 1, 0 for LSB
    LDR R2, =0x80000002 // set bits 31, 30 for MSB and bits 1, 0 for LSB to 10 (pull down)
    ORR R1, R1, R2 // so that we can safely depict 0 (GND) when the buttons (leds) are not pressed
    STR R1, [R0, #PUPDR_OFFSET]


    LDR R0, =GPIOA_BASE // R0 used for the output
    LDR R1, =GPIOD_BASE // R1 used for the input
    MOV R2, #0x0001 // R2 used for the moving led location, initialized to 1 (LSB led on)

main_loop:
    LDR R4, [R1, #IDR_OFFSET]   // read input from IDR GPIOD

    TST R4, #0x0001             // test if LSB led is on
    BEQ write_led               // if not, do not move the output led

    LDR R5, =0x8000             // if yes, check the MSB for direction
    TST R4, R5                  // if it is on
    BNE move_right              // move to the right

move_left:
    LSL R2, R2, #1              // else move to the left
    LDR R5, =0x10000            // 17-th bit used to check overflow
    CMP R2, R5                  // if bit overflow happened
    BNE write_led               // jump to led depiction if not
    MOV R2, #0x0001             // go to the LSB again
    B write_led                 // and then jump to led depiction
```

```
move_right:
    LSR R2, R2, #1              // move to the right
    CMP R2, #0                  // check for overflow
    BNE write_led               // jump to led depiction if not
    LDR R2, =0x8000             // go to the MSB again

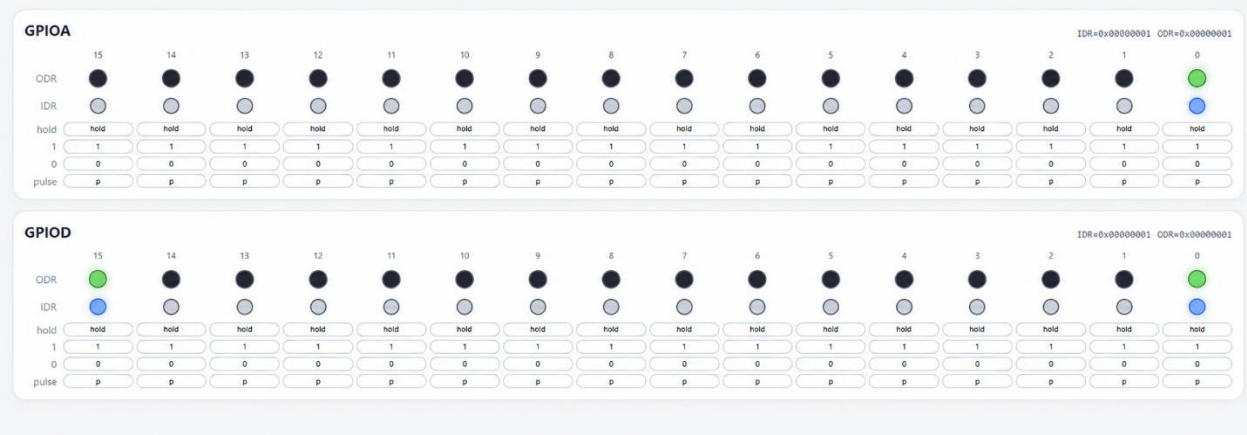
write_led:
    STR R2, [R0, #ODR_OFFSET]   // write R2 content (led location) to the ODR

    LDR R3, =0x00FFFFFF         // use delay so that the led light changing location is visible
delay_loop:
    SUBS R3, R3, #1             // decrement (with CPSR change) the delay timer
    BNE delay_loop

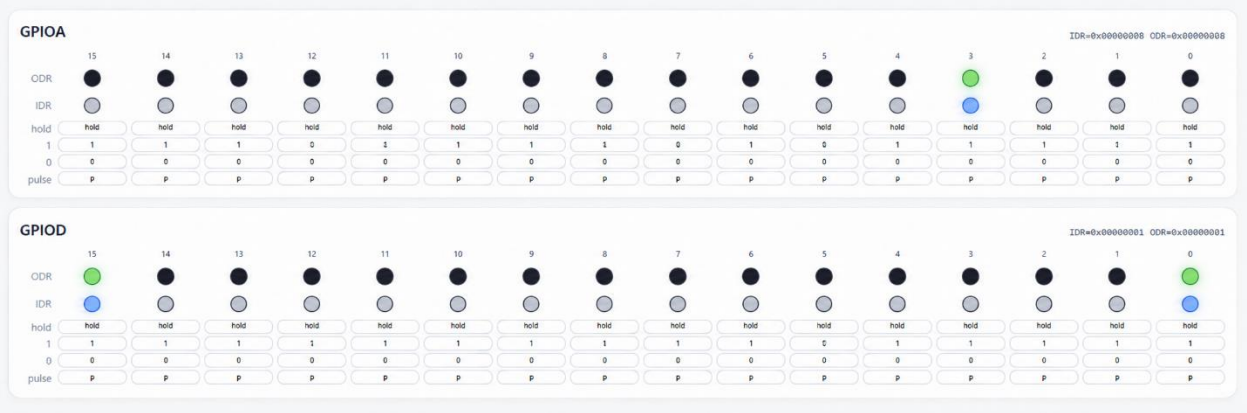
    B main_loop                 // repeat when delay timer is zero

```

Παρατηρούμε ότι για αναμμένα τα LED MSB και LSB στο GPIOD:



Μετά από λίγο το LED (13 βήματα αργότερα), βρίσκεται στη θέση 3 στο GPIOA, κινούμενο δεξιόστροφα, και μάλιστα ο κώδικας χειρίζεται το overflow μετά τη θέση 0 στην αρχή, σωστά.



3η Άσκηση:

α) Ο κώδικας που υλοποιεί το ζητούμενο του πρώτου ερωτήματος είναι ο ακόλουθος (αποτελεί το 1ο κομμάτι ενιαίου κώδικα για την εκτέλεση όλων των ερωτημάτων)

```
.syntax unified
.thumb
.text
.global asm_main_
.type asm_main_, %function
.align 2

.equ RCC_BASE,   0x40023800   // Reset and Clock Control
.equ GPIOD_BASE, 0x40020C00   // GPIOD base (Output)

.equ RCC_AHB1ENR, 0x30        // RCC AHB 1 enable
.equ MODER_OFFSET,  0x00      // Mode Register
.equ OTYPER_OFFSET, 0x04      // Output Type Register
.equ ODR_OFFSET,    0x14      // Output Data Register

asm_main_:
    LDR R0, =0x20000900 // initial address
    MOV R1, #0x000000FF // initial number (32 bits, word size)

loop_store:
    STR R1, [R0], #4 // post-index address increment by word
    SUBS R1, R1, #1
    CMP R1, #0
    BGE loop_store   // if less than zero, only then break (zero included)

```

Μπορούμε να εξακριβώσουμε την ορθότητα του προγράμματος, ελέγχοντας μέσω του GUI του Renode, τα περιεχόμενα της μνήμης στις θέσεις 0x20000900, 0x20000904 και 0x20000CFC (τελευταία θέση τοποθέτησης δεδομένων), τα οποία είναι τα αναμενόμενα.

```
>>> sysbus ReadDoubleWord 0x20000900
sysbus ReadDoubleWord 0x20000900
0x000000FF
[33;1m(machine-0) [0m
>>> sysbus ReadDoubleWord 0x20000904

>>> sysbus ReadDoubleWord 0x20000904
sysbus ReadDoubleWord 0x20000904
0x000000FE
[33;1m(machine-0) [0m
>>> sysbus ReadDoubleWord 0x20000CFC

>>> sysbus ReadDoubleWord 0x20000CFC
 [0msysbus ReadDoubleWord 0x20000CFC
0x00000000
```

β) Ο κώδικας που υλοποιεί το ζητούμενο του δεύτερου ερωτήματος είναι ο ακόλουθος (αποτελεί το 2ο κομμάτι ενιαίου κώδικα για την εκτέλεση όλων των ερωτημάτων)

```
    LDR R0, =0x20000900 // initial address
    MOV R2, #0          // zero counter
    MOV R4, #256        // numbers counter

loop_zero_count:
    MOV R3, #32         // bits left counter (32 bits for word)
    LDR R1, [R0], #4    // post-index address increment by word

single_number_zero_count:
    LSRS R1, R1, #1     // put LSB to C
    IT LO
    ADDLO R2, R2, #1    // if C = 0, increment zero counter
    SUBS R3, R3, #1     // decrement bits left counter
    BNE single_number_zero_count // if it is zero, we have included all bits

check_completion:
    SUBS R4, R4, #1     // decrement numbers counter
    BNE loop_zero_count // include zero

output_1:
    LDR R0, =RCC_BASE
    LDR R1, [R0, #RCC_AHB1ENR]
    ORR R1, R1, #0x08           // set bit 3 of RCC_AHB1ENR
    STR R1, [R0, #RCC_AHB1ENR]  // we enable the clock for GPIOD

    LDR R0, =GPIOD_BASE

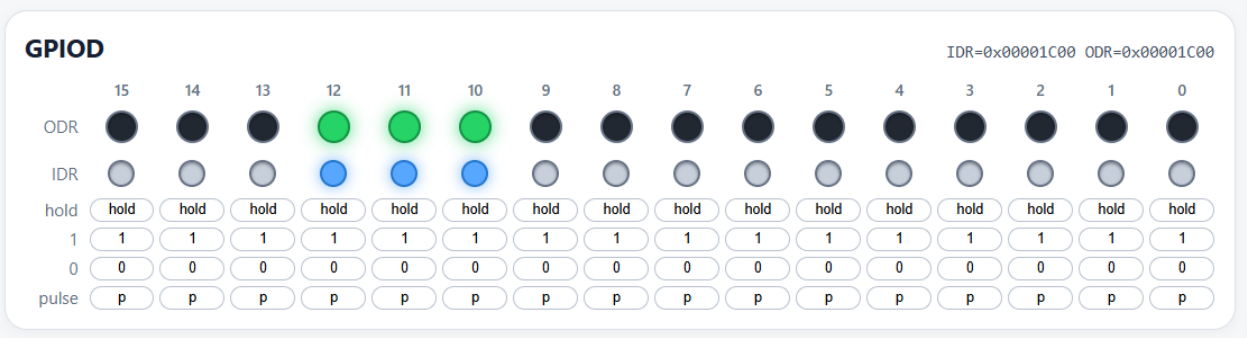
    LDR R1, =0x55555555 // set bits for all 16 pins to 01
    STR R1, [R0, #MODER_OFFSET] // so that they are used for the output

    MOV R1, #0x0000 // set bits for all 16 pins to 0
    STR R1, [R0, #OTYPER_OFFSET] // so that we have push-pull output

    STR R2, [R0, #ODR_OFFSET] // write R2 content (led location) to the ODR


```

Μπορούμε να εξακριβώσουμε την ορθότητα του προγράμματος, ελέγχοντας μέσω του GUI του Renode, τα LED που είναι αναμμένα, δηλαδή τα LED στις θέσεις 10, 11 και 12. Στις 256 32-bit θέσεις μνήμης έχουμε 32 * 256 = 8192 bits συνολικά, εκ των οποίων, τα 1024 (τα μισά από τα 2048 συνολικά bits για 8-bit καταχωρητές, όπως είδαμε στην 1^{η} σειρά ασκήσεων) είναι μηδενικά για τα τελευταία 8 bits κάθε καταχωρητή, ενώ όλα τα υπόλοιπα 24 bits είναι μηδέν για όλους τους αριθμούς, επιπλέον 24 * 256 = 6144 μηδενικά. Έτσι, συνολικά έχουμε 7168 μηδενικά, 0001110000000000, σε δυαδική αναπαράσταση 16 bit, όπως φαίνεται και στην ακόλουθη εικόνα.



γ) Ο κώδικας που υλοποιεί το ζητούμενο του τρίτου ερωτήματος είναι ο ακόλουθος (αποτελεί το 3ο κομμάτι ενιαίου κώδικα για την εκτέλεση όλων των ερωτημάτων)

```
    LDR R0, =0x20000900 // initial address
    MOV R2, #0          // numbers between counter
    MOV R4, #256        // numbers counter

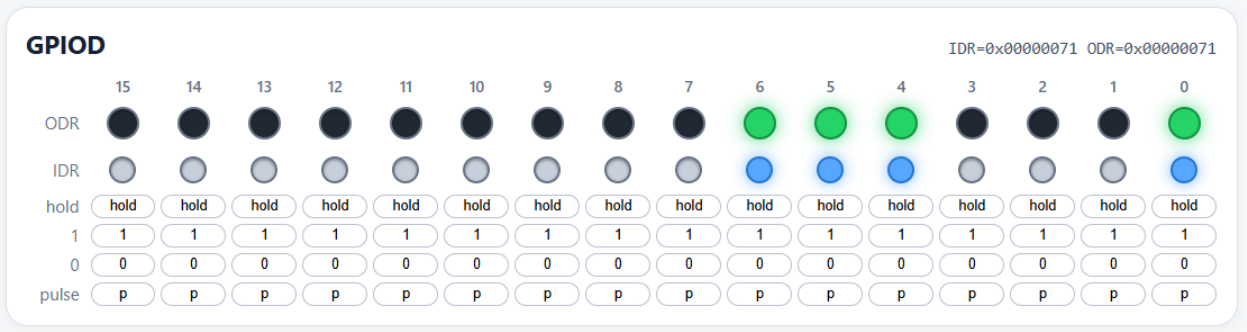
loop_between_count:
    LDR R1, [R0], #4    // post-index address increment by word
    CMP R1, #0x10       // 1st comparison
    BLT check_finish    // if not satisfied, next number
    CMP R1, #0x80       // 2nd comparison
    IT LE
    ADDLE R2, R2, #1    // if both are satisfied, include this number

check_finish:
    SUBS R4, R4, #1     // decrement numbers counter
    BNE loop_between_count  // include zero

output_2:
    LDR R0, =GPIOD_BASE
    STR R2, [R0, #ODR_OFFSET] // write R2 content (led location) to the ODR

endless_loop:
    B endless_loop
```

Μπορούμε να εξακριβώσουμε την ορθότητα του προγράμματος, ελέγχοντας μέσω του GUI του Renode, τα LED που είναι αναμμένα, δηλαδή τα LED στις θέσεις 0, 4, 5, 6. Το πλήθος των αριθμών μεταξύ 0-255 που βρίσκονται στο διάστημα [0x10, 0x80] = [16, 128] είναι 128 – 16 + 1 = 113, 0000000001110001 σε δυαδική αναπαράσταση 16 bit, όπως φαίνεται και στην ακόλουθη εικόνα.



δ) Η διαφορά που παρατηρώ σε σχέση με την αντίστοιχη 1η άσκηση της 1ης σειράς ασκήσεων για τον AVR, πέρα από τις διαφορές στη φιλοσοφία των δύο αρχιτεκτονικών, είναι το διαφορετικό αποτέλεσμα στο ερώτημα β). Ενώ στην προηγούμενη σειρά ασκήσεων το πλήθος των μηδενικών ήταν 1024, στην άσκηση αυτή προκύπτει 7168. Η διαφορά αυτή οφείλεται στο ότι ζητήθηκε κάθε αριθμός να αποθηκεύεται σε 32 bit. Έτσι, πέρα από τα 8 λιγότερο σημαντικά bits που περιέχουν την πραγματική πληροφορία των αριθμών 0–255, τα 24 πιο σημαντικά bits κάθε λέξης είναι μηδενικά και συνεισφέρουν επιπλέον μηδενικά στο συνολικό αποτέλεσμα. Αν χρησιμοποιούνταν μόνο τα 8 τελευταία bits ή αν πολλοί αριθμοί συσκευάζονταν στην ίδια λέξη μνήμης, οι δύο ασκήσεις θα οδηγούσαν στο ίδιο πλήθος μηδενικών. Έτσι, ο πλεονασμός των 24 πιο σημαντικών bit κάθε λέξης αποτυπώνεται στο αποτέλεσμα, το οποίο είναι σημαντικά μεγαλύτερο.

Θεωρητικές ασκήσεις:

4η Άσκηση:

α) Για τα παραπάνω use cases, θα χρησιμοποιήσουμε τους ακόλουθους ARM επεξεργαστές:

Automotive systems controller: **ARM Cortex R5,** διότι προσφέρει real time performance με functional safety, δηλαδή είναι κατάλληλος για λειτουργικότητες που απαιτούν αστραπιαία απόκριση, αλλά και εξασφάλιση ορθής λειτουργίας όπως το άνοιγμα των αερόσακων.

Drone path planning with ROS (Robot Operating System): **ARM Cortex A15**, καθώς προσφέρει υψηλή απόδοση και είναι κατάλληλος για την

υποστήριξη πλήρους λειτουργικού συστήματος, όπως το συγκεκριμένο multi-threaded ROS του drone.

Weather Base Station: **ARM Cortex M0**, εφόσον απαιτείται υψηλή αυτονομία και ο M0 είναι ο πιο ενεργειακά αποδοτικός και λιγότερο κοστοβόρος επεξεργαστής από τους 4.

IoT for Health: **ARM Cortex M4**, αφού δίνεται περισσότερη σημασία στην απόδοση του επεξεργαστή, και συγκεκριμένα σε DSP, από την ενεργειακή του αποδοτικότητα και την αυτονομία του, χωρίς αυτό να σημαίνει ότι είναι αδιάφορη. Ο ARM Cortex M4, επιτυγχάνει τη βέλτιστη ισορροπία επίδοσης-αυτονομίας.

β) Η τεχνολογία big.LITTLE επιτυγχάνει καλύτερη απόδοση των επεξεργαστών σε διαφορετικές συνθήκες φόρτου εργασίας (bursty, normal), ομαδοποιώντας κατάλληλα τους αποδοτικότερους ενεργειακά με τους υψηλότερους σε επίδοση, ώστε να εξυπηρετείται κάθε είδος κίνησης το ταχύτερο δυνατό, αλλά και να εξοικονομείται η μέγιστη δυνατή ποσότητα ενέργειας.

Το μοναδικό use case που έχει διαφορετικές συνθήκες φόρτου εργασίας, είναι το Drone path planning with ROS (Robot Operating System), διότι οι 2 ζητούμενες λειτουργίες του απαιτούν η $1^{η}$ ανάγνωση σημάτων από αισθητήρες (χαμηλός φόρτος) και η $2^{η}$ την υλοποίηση multi-threaded λειτουργικού συστήματος (υψηλός φόρτος). Τα άλλα 3 use cases είτε απαιτούν μόνο χαμηλό φόρτο εργασίας (Weather Base Station), είτε μόνο bursty φόρτο εργασίας (Automotive systems controller, IoT for Health), επομένως η τεχνολογία big.LITTLE δε θα εξυπηρετούσε τις ανάγκες των εφαρμογών.

$5^{η}$ Άσκηση:

Δεδομένου ότι η αρχική κατάσταση των καταχωρητών είναι:
PC = 0x00008000

SP = 0x00010000

r0 = 0x00000000

r1 = 0x00000000

r2 = 0x00000000

r3 = 0x00000000

r4 = 0x00000000

r5 = 0x00000000

r9 = 0x00000000

CPSR = 0x00000013 (SVC mode, entered after reset),

η κατάσταση των καταχωρητών σε κάθε ένα από τα checkpoints θα είναι:

CHK1:

PC = 0x00008010 (λαμβάνοντας υπόψη το 3 stage pipeline, χωρίς αυτό -8 H)

SP = 0x00010000

r0 = 0x80000000

r1 = 0x00000000

r2 = 0x00000000

r3 = 0x00000000

r4 = 0x00000000

r5 = 0x00000000

r9 = 0x00000000

CPSR = 0x70000013 (N=0, Z=1, C=1, V=1),

CHK2:

PC = 0x00008014 (λαμβάνοντας υπόψη το 3 stage pipeline, χωρίς αυτό -8 H)

SP = 0x0000FFF8

r0 = 0x80000000

r1 = 0x00000000

r2 = 0x00000000

r3 = 0x00000000

r4 = 0x00000000

r5 = 0x00000000

r9 = 0x00000000

CPSR = 0x70000013 (N=0, Z=1, C=1, V=1),

CHK3:

PC = 0x00008028 (λαμβάνοντας υπόψη το 3 stage pipeline, χωρίς αυτό -4 H)

SP = 0x0000FFF8

r0 = 0x80000000

r1 = 0x00000000

r2 = 0x00000000

r3 = 0x00008025 (thumb_part + 1, το οποίο +1 αφαιρείται από την BX και T=1)

r4 = 0x00000000

r5 = 0x00000000

r9 = 0x00000000

CPSR = 0x80000033 (N=1, Z=0, C=0 (borrow used for sub), V=0, T=1),

CHK4:

PC = 0x00008034 (λαμβάνοντας υπόψη το 3 stage pipeline, χωρίς αυτό -8 H)

SP = 0x00010000

r0 = 0x80000000

r1 = 0x00000000

r2 = 0x00000000

r3 = 0x0000802C (arm_back)

r4 = 0x80000000

r5 = 0x00000000

r9 = 0x00000000

CPSR = 0x60000013 (N=0, Z=1, C=1 (borrow not used for sub), V=0, T=0).